

Retrieval-Augmented Generation

Applied Deep Learning — Chapter 12

Yin Yang

Foundation Books Project

Roadmap

The Running Example

Documents, Chunks, and Metadata

Lexical Retrieval Baseline

Dense Retrieval

Vector Search at Scale

Hybrid Search and Reranking

Building the LLM Context

Evaluating RAG Systems

Failure Modes and Safety

Practical Recipe and Summary

The Running Example

A Study Assistant That Cites

Learner asks:

How can I improve validation accuracy without leaking test data?

- Prompt-only LLM: plausible advice, may miss the stated rule.
- Study corpus contains a chunk that names exactly the validation rule.
- RAG: retrieve that chunk **before** generation; cite it in the answer.

The difference is not polish — it is that the answer is connected to a visible source.

Corpus collection of source documents the system can search.

Query the user's information need as represented for retrieval.

Grounded answer generated answer whose factual claims are supported by retrieved chunks.

Prompt-only baseline generated answer with *no* retrieved context — the first comparison.

What RAG Changes (and What It Does Not)

Prompt-only fix

- Change wording or decoding.
- Right info, wrong format.

RAG fix

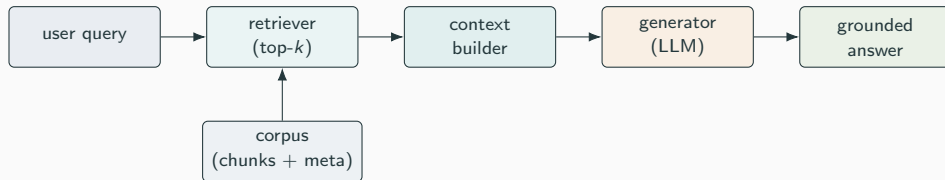
- Change information at inference.
- Missing sources, local policy.
- Weights stay fixed.

Fine-tuning fix

- Change weights.
- Repeated behavior shift.
- Slowest, most complex.

Order: prompt-only → RAG → fine-tuning.

A Minimal RAG System



Four pieces: **corpus**, **retriever**, **context-builder**, **generator**. Plus citations: pointers from sentences back to chunk IDs.

Documents, Chunks, and Metadata

Chunking: Why Smaller Pieces

Even with long context, reading the whole textbook per question is slow, expensive, and noisy.

Chunk searchable piece of a source document.

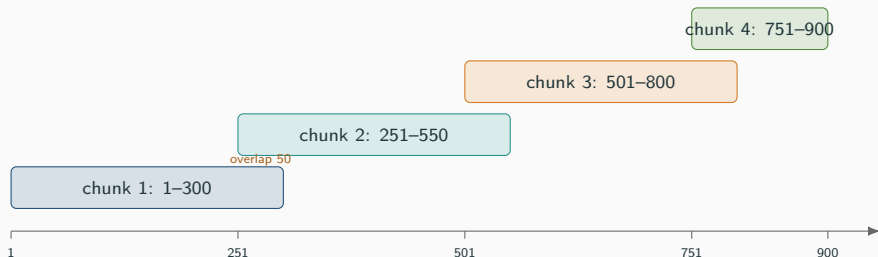
Chunk size tokens / words / chars per chunk.

Overlap repeated text shared by neighbors — protects against boundary failures.

Tradeoff: tiny chunks lose context; huge chunks dilute retrieval scores and waste prompt budget.

Chunking Arithmetic

Document: 900 tokens, chunk size 300, overlap 50; stride = 250.



Chunking is a modeling choice, not boilerplate preprocessing.

Metadata: Make Retrieval Auditable

Metadata is structured info attached to each chunk: title, chapter, section, source path, date, language, split, license, corpus version.

doc_id	chunk_id	metadata	excerpt
train	c12	ch. 6 validation	"Use the validation set for selection. . ."
emb	c04	ch. 8 embeddings	"Normalize before cosine retrieval. . ."
llm	c09	ch. 11 decoding	"Keep decoding fixed when comparing prompts. . ."

Enables filtering, citation, refresh, error analysis. **Secrets do not belong in metadata.**

Lexical Retrieval Baseline

Why Start Lexical

- Fast and transparent.
- Often strong when query and source share terminology.
- Concrete debugging baseline for any later neural retriever.

Inverted index: term $\rightarrow$ chunks containing it. Term validation $\rightarrow$ train:c12, nnt:c07, 11m:c10. Crude overlap counts ignore term rarity, repetition, document length.

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1(1 - b + b|d|/\overline{|d|})}.$$

- IDF rewards **rare** terms.
- Numerator/denominator **saturates** repeated terms.
- $|d|$ vs. average length: **normalizes for document size**.
- k_1, b : tunable constants.

Lexical fails on paraphrases: “leaderboard” vs. “final test set.”

Dense Retrieval

Embedding-Based Retrieval

Encoder maps query and chunks to vectors; rank by similarity.

$$s_{\text{dot}}(q, j) = e_q^\top e_j, \quad s_{\text{cos}}(q, j) = \frac{e_q^\top e_j}{\|e_q\|_2 \|e_j\|_2}.$$

Top- k :

$$\text{TopK}_k(q) = \arg \operatorname{topk}_j s(q, j).$$

Returned IDs (not just scores) are what make citation possible.

Definition

Fraction of held-out questions for which an acceptable source appears in the top- k results.

Example: 8 of 10 questions retrieve a needed source in the top 5.

$$\text{recall@5} = 0.8.$$

Dense vs. lexical:

- Dense: handles paraphrase, may miss rare names / code identifiers.
- Lexical: nails exact phrases, misses paraphrase.

An index record must include enough to keep similarity meaningful:

- encoder name and version
- embedding dimension and normalization rule
- distance metric (dot product / cosine)
- preprocessing rule (lowercasing, truncation)
- corpus version
- chunk and document IDs

Stored vectors and query vectors must come from the same model. Otherwise scores look valid while ranking nonsense.

Vector Search at Scale

When Exact Search Hurts

10 M chunks, dim 768:

$$10^7 \times 768 = 7.68 \times 10^9$$

multiply-adds per query for exact scan. Raw float32 vectors:

$$10^7 \times 768 \times 4 \approx 30.7 \text{ GB.}$$

ANN search approximate nearest-neighbor: small chance of missing the exact best, big speedup.

Vector database storage + ANN + metadata filters + refresh.

Reference Scaling Numbers

Vectors	Exact p50	Approx p50	Approx recall@10
10,000	0.120 ms	0.184 ms	0.916
50,000	0.905 ms	1.147 ms	0.931
100,000	1.795 ms	3.055 ms	0.963

- At small scale, exact scan can *beat* a Python-level approx index.
- Index choice should be measured at the actual scale, hardware, and filter pattern.
- The right chunk is useless if it arrives after the latency budget is gone.

Vector DBs Don't Make Embeddings Strong

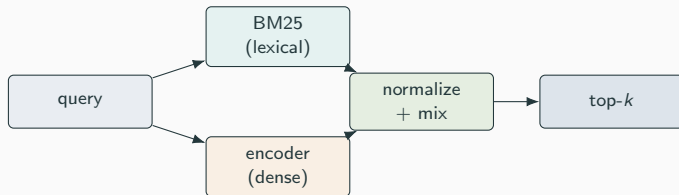
Quality still comes from:

- source corpus
- chunking rule
- encoder, normalization, metric
- metadata
- evaluation set

Metadata filters are often as important as scores: restrict to assignment instructions before ranking, or to current corpus version after a policy change. **Index refresh** is required when corpus, chunking, encoder, or preprocessing changes.

Hybrid Search and Reranking

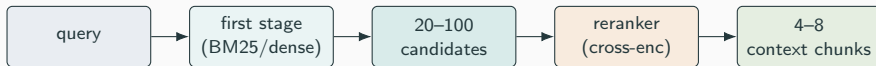
Hybrid: Lexical + Dense



Score-level hybrid:

$$s_{\text{hybrid}}(q, d) = \lambda \tilde{s}_{\text{BM25}}(q, d) + (1 - \lambda) \tilde{s}_{\text{dense}}(q, d).$$

Set-level hybrid: union of BM25 top- k and dense top- k , then rank.



Cross-encoder reads query and one chunk together; pair score.

Late interaction (ColBERT) keeps token-level matching cheap enough to scale.

A reranker can only reorder the candidate set. It cannot recover evidence the first stage missed.

Reranking Cost

20 ms per query-chunk pair, scoring 50 candidates one at a time:

$$50 \times 20 \text{ ms} \approx 1 \text{ s.}$$

Mitigations:

- batched scoring
- smaller reranker model
- shrink candidate pool

Adopt reranking only after lexical and dense baselines are visible.

Building the LLM Context

Context Budget Arithmetic

Window 4096 tokens; system prompt 220; user question 60; reserve 500 for answer + safety:

$$4096 - 220 - 60 - 500 = 3316 \text{ tokens for retrieved chunks.}$$

Average chunk 350 tokens:

$$\lfloor 3316/350 \rfloor = 9 \text{ chunks max.}$$

Realistic: 7–8 chunks to leave separators, citation labels, and edges. **Context is scarce. Reserve before stuffing.**

Prompt Template Sketch

- **System:** “Answer using the retrieved sources when relevant. If they don’t support the claim, say what is missing. Cite chunk IDs in square brackets.”
- **Question:** “How can I improve validation accuracy without leaking test data?”
- **Sources:**
 - [train:c12] Use validation for selection. Don’t tune on the final test set.
 - [nnt:c07] When accuracy changes, record factor, time, metric, and an example.

Citation IDs are the receipt for each claim the answer is allowed to make.

Top- k Sweep, Held Everything Else Fixed

Chunks	Mean prompt tokens	Answer score	Median latency
1	239	3.500	0.968 s
3	390	4.750	1.190 s
5	538	4.000	0.925 s
8	758	4.000	1.232 s

More text $\neq$ better. Top-3 won; extra chunks added cost without quality, and didn't fix the model's weak citation copying.

Definition

An *abstention rule* tells the system when to decline or to say that retrieved sources are insufficient.

- Quality target: an unsupported answer should not be rewarded merely because it is fluent.
- Deduplicate near-identical chunks.
- Preserve source IDs through the prompt.
- Record the ordering rule — changing order changes answers.

Evaluating RAG Systems

Two Different Questions

Retrieval evaluation: did we find useful evidence?

Answer evaluation: did the generator use it correctly? Combining them into “the answer sounds good” makes debugging impossible.

Faithfulness is the answer supported by the retrieved sources?

Citation correctness do the cited chunks support the claims?

Recall@k and MRR

Five questions; first acceptable source at ranks 1, 2, none, 3, 1.

$$\text{recall@5} = 4/5 = 0.80.$$

Mean reciprocal rank:

$$\text{MRR} = \frac{1 + 1/2 + 0 + 1/3 + 1}{5} = \frac{2.833}{5} \approx 0.567.$$

These are **retrieval** numbers only — they say nothing about whether the generator wrote a faithful answer.

A Reference RAG Run

Eight held-out questions, top-3 retrieval, small Qwen2.5-0.5B-Instruct generator.

Run	Recall@3	MRR	Answer score
Prompt-only	–	–	2.875
Lexical RAG	0.875	0.750	4.125
MiniLM dense RAG	0.875	0.729	3.875
Hybrid RAG	0.875	0.792	4.750

Hybrid lifted the score; the small generator still often dropped citation IDs — a generation failure, not a retrieval failure.

Three different latencies:

- retrieval (search + rerank)
- generation (LLM call)
- full answer (what the user feels)

Improving 6.8 $\rightarrow$ 8.1/10 while going 0.8s $\rightarrow$ 8.0s may or may not be acceptable. Report all of them, plus prompt token count, chunks in context, faithfulness, citations.

RAG can leak just like ordinary ML pipelines:

- a test answer copied into the corpus
- final questions guiding chunk-size or top- k choices
- vector index quietly including held-out sources

Validation questions pick chunk size, top- k , encoder, reranker. **Test questions** run *after* the recipe is selected.

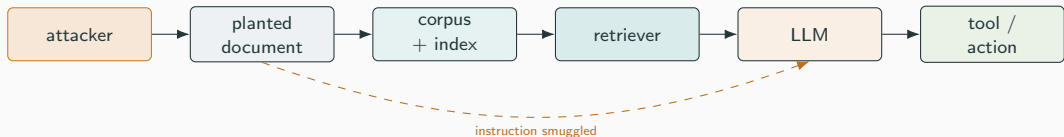
Failure Modes and Safety

Stage-by-Stage Debugging

Stage	Symptom	First check
Corpus	policy never found	Is the document in the corpus, parsed correctly?
Chunking	answer split across chunks	Inspect boundaries near the gold source.
Retrieval	gold source absent from top- k	Compare lexical, dense, hybrid.
Reranking	gold source dropped	Inspect candidate ranks before/after.
Context	gold source truncated	Count prompt tokens; check ordering.
Generation	unsupported claims	Compare answer sentences against cited chunks.
Safety	retrieved text gives instructions	Treat sources as data; reduce action surface.

Assign each failure to the **earliest stage that could cause it**.

Indirect Prompt Injection



- Attacker never talks to the model — only plants retrievable text.
- “Sources:” labels help but are **not a security boundary**.
- Limit allowed actions; treat generated actions as untrusted.

Practical Recipe and Summary

A Disciplined First RAG System

1. Freeze held-out questions with expected source IDs.
2. Run a prompt-only baseline (fixed model, prompt, decoding).
3. Parse documents, inspect chunks; record size, overlap, metadata.
4. Run lexical and dense retrieval; record $\text{recall}@k$, latency, encoder, dimension, metric, top- k , index version.
5. Add one improvement: chunk size, top- k , hybrid, or reranking.
6. Generate cited answers with fixed decoding; score retrieval, answers, faithfulness, citations, latency, failure classes.

Required Artifacts

Every RAG report should commit:

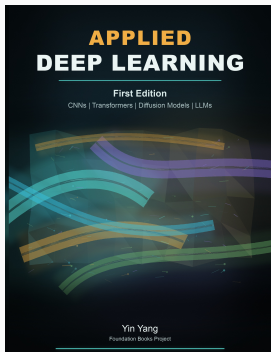
- the question set with gold source IDs
- the chunk table with metadata and corpus version
- top- k retrieval outputs per query
- prompts with retrieved context attached
- generated answers and their citations
- scored errors classified by stage

These tell where the system improved, where it slowed, where it still fails.

Summary

- RAG = adds an *external evidence* step before generation; weights stay fixed.
- Pipeline: corpus $\rightarrow$ chunks + metadata $\rightarrow$ retriever $\rightarrow$ context $\rightarrow$ generator $\rightarrow$ cited answer.
- Lexical baseline first; dense for paraphrase; hybrid + rerank for both.
- Context is scarce: budget tokens before chunks.
- Score retrieval and answer quality *separately*; report latency too.
- Retrieved text is untrusted; abstention and error analysis are part of quality.
- Use RAG when *evidence* is missing; use fine-tuning when *behavior* needs to change repeatedly.

**Next: when fixed-weight retrieval cannot fix the failure,
adapt the weights efficiently — LoRA and QLoRA.**



Applied Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	foundation-books.github.io/applied-deep-learning
Code	github.com/foundation-books/applied-deep-learning-code
Print	amazon.com/dp/B0GZF7QRSH
Kindle	amazon.com/dp/B0GZF9221X
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.